Configuration Issues & Solutions

Technical Analysis of Database Migrations, Permission Grants, Environment Parsing, and Local Admin Provisioning

Executive Summary

During local setup and initial code configuration for the application environment, four distinct technical roadblocks were identified and resolved. These issues spanned database table creation, PostgreSQL security roles, environment variable file formatting, and user authorization/module activation. This document details the root causes, diagnostic steps, and exact resolution commands applied to achieve a fully operational environment.

Issue Area	Root Cause	Resolution Mechanism	Status
Database Schema & Tables	Unexecuted Alembic migrations & missing tables (e.g. local_import_settings).	Executed Alembic upgrade head on main & control plane DBs + tenant bootstrap & seeding.	**Resolved**
Database Role Privileges	Application role (hr_app) lacked SELECT/INSERT/UPDATE permissions on public schema.	Created & executed grant_app.py to grant DB connect, schema usage, table CRUD & sequence rights.	**Resolved**
Environment Parsing (.env)	Missing newline preceding SECRET_BACKEND= in .env corrupting setting values.	Ran Python regex script to auto-fix formatting and reload environment configs.	**Resolved**
Admin Privileges & Modules	Default tenant user lacked Super Admin IAM rights; modules toggled off.	Executed dev_make_admin.py to assign Super Admin role and activate all system modules.	**Resolved**

Table of Contents

(Note: The Table of Contents above will automatically populate when opened in Microsoft Word or Google Docs.)

1. Missing Database Tables & Unapplied Migrations

The application encountered immediate runtime failures when executing database queries against required configuration tables such as local_import_settings. SQL errors indicated that relation tables did not exist in the public schema.

Symptom & Error Details

When attempting to fetch import settings during application startup or local background scanning, the ORM threw a SQL error:

```
[SQL: SELECT local_import_settings.id AS local_import_settings_id, ... FROM
local_import_settings WHERE local_import_settings.id = %(pk_1)s AND
local_import_settings.tenant_id = %(tenant_id_1)s::UUID]
```

Root Cause Analysis

The failure was caused by a two-fold configuration gap:
1. Schema migrations managed by Alembic had not been executed against the target PostgreSQL database.
2. Misalignment between the migration connection string (DATABASE_URL) used by administrative superusers and the runtime application connection string (APP_DATABASE_URL).

Resolution Steps & Executed Commands

To establish the database schema and initialize required baseline data, the following workflow was executed:

- 1. Connection URL Alignment:

Configured DATABASE_URL for administrative superuser migrations and APP_DATABASE_URL for standard application access:

```
DATABASE_URL=postgresql+psycopg2://postgres:...@localhost:5432/hr_automation
APP_DATABASE_URL=postgresql+psycopg2://hr_app:...@localhost:5432/hr_automation
```

- 2. Database Migration Execution:

Applied Alembic migrations across both the primary application database and the control plane database:

```
.venv\Scripts\python -m alembic current
.venv\Scripts\python -m alembic upgrade head
python -m alembic -c alembic_control_plane.ini upgrade head
```

- 3. Tenant Initialization & Seeding:

Bootstrapped the initial tenant and populated essential master data, pipeline configurations, and workflow records:

```
python -m hrctl.installation init
python -m hrctl.tenant bootstrap-existing --slug ctd-techs --name "CTD Techs"
python -m hrctl.seed_masters
python -m hrctl.seed_workflows
python -m hrctl.seed_pipeline
```

Verification Outcome
Running table checks via python verified that local_import_settings was successfully created under public.local_import_settings and populated with total table counts.

2. Application Database Role Permission Grants

After successfully creating database tables using the postgres superuser, the application runtime (operating under the hr_app database user) was unable to read or write to the newly migrated tables due to insufficient permission grants.

Root Cause Analysis

By default, PostgreSQL tables created during migrations belong exclusively to the executing role (postgres). The separate application user (hr_app) specified in APP_DATABASE_URL lacked CONNECT, USAGE, and standard CRUD privileges on the public schema objects and auto-incrementing sequences.

Resolution Steps & Permission Script

A dedicated Python utility grant_app.py was authored and executed using administrative privileges to grant complete runtime permissions to the hr_app role across both primary and control plane databases.

```
from urllib.parse import urlparse
from sqlalchemy import create_engine, text
from app.core.config import settings as s

role = urlparse(s.APP_DATABASE_URL).username
for url in (s.DATABASE_URL, s.CONTROL_PLANE_DATABASE_URL):
    db = urlparse(url).path.lstrip("/")
    with create_engine(url, isolation_level="AUTOCOMMIT").connect() as c:        for
q in (
            f'GRANT CONNECT ON DATABASE "{db}" TO "{role}"',
            f'GRANT USAGE ON SCHEMA public TO "{role}"',
            f'GRANT SELECT, INSERT, UPDATE, DELETE ON ALL TABLES IN SCHEMA public TO
"{role}"',
            f'GRANT USAGE, SELECT ON ALL SEQUENCES IN SCHEMA public TO "{role}"',
            f'ALTER DEFAULT PRIVILEGES IN SCHEMA public GRANT SELECT, INSERT, UPDATE,
DELETE ON TABLES TO "{role}"',
            f'ALTER DEFAULT PRIVILEGES IN SCHEMA public GRANT USAGE, SELECT ON
SEQUENCES TO "{role}"',
        ):
```

```
            c.execute(text(q))
        print("granted on", db, "to", role)
```

Execution Command:

```
python grant_app.py
```

3. Environment Variable Formatting & Parsing Errors

During configuration inspection of control plane URLs and encryption backend settings, configuration parsers failed to read CONTROL_PLANE_DATABASE_URL and APP_CONTROL_PLANE_DATABASE_URL correctly.

Root Cause Analysis

Inspection of the local .env configuration file revealed that a line break was missing prior to SECRET_BACKEND=db_encrypted. As a consequence, the variable key SECRET_BACKEND was directly appended to the end of the preceding database URL string, corrupting both values during application settings parsing.

Resolution Steps & Automated Sanitization

An inline Python regex script was executed to programmatically repair the formatting in .env:

```
.venv\Scripts\python -c "import re,pathlib; p=pathlib.Path('.env');
t=p.read_text(encoding='utf-8'); t=re.sub(r'(?<!\n)SECRET_BACKEND=',
'\nSECRET_BACKEND=', t); p.write_text(t, encoding='utf-8'); print('fixed')"
```

Following the repair, environment variables were validated and re-verified:

```
set | findstr /i "DATABASE_URL SECRET_BACKEND"
.venv\Scripts\python -c "from app.core.config import settings as s;
print(s.CONTROL_PLANE_DATABASE_URL.rsplit('/',1)[1]);
print(s.APP_CONTROL_PLANE_DATABASE_URL.rsplit('/',1)[1])"
```

The installation init and tenant bootstrap processes were then cleanly re-run:

```
.venv\Scripts\python -m hrctl.installation init
.venv\Scripts\python -m hrctl.tenant bootstrap-existing --slug ctd-techs --name "CTD
Techs"
.venv\Scripts\python -m hrctl.seed_masters
.venv\Scripts\python -m hrctl.seed_workflows
.venv\Scripts\python -m hrctl.seed_pipeline
```

4. Local User Admin Rights & Module Activation

Upon logging into the local development environment, newly created user accounts were unable to access administrative endpoints or interact with platform modules because their default role was non-administrative and all optional modules were turned off.

Root Cause Analysis

Standard tenant creation processes seed basic tenant structures but leave individual user role assignments as standard 'user'. Additionally, application modules registered under the platform module registry remain inactive until explicitly toggled on.

Resolution Steps & Development Script

A developer utility script dev_make_admin.py was created to bridge standard user privileges to Super Admin and activate all available platform modules for the specified local company tenant.

```
"""DEV ONLY: make one user Super Admin and switch on every module for the local
company."""
import sys
import hrctl  # registers models + tenant hooks
from app.core.database import SessionLocal
from app.models.user import User
from app.platform import module_registry as reg, module_status
from app.platform.iam.roles import bridge_single_user
from app.platform.tenancy.context import TENANT_1_ID, TenantContext, tenant_context

email = sys.argv[1].strip().lower()
tenant = TenantContext(tenant_id=TENANT_1_ID, slug="ctd-techs",
isolation="dedicated_db")
with tenant_context(TENANT_1_ID):
    db = SessionLocal()
    try:
        user = db.query(User).filter(User.email == email).first()
        if user is None:
            sys.exit(f"No user with email {email}")
        user.role = "admin"
        db.commit()
        bridge_single_user(db, tenant, user)  # adds the Super Admin role
        done = module_status.activate(db, tenant, sorted(reg.CHOOSABLE), actor="dev-
setup")
        db.commit()
        print("Super Admin:", user.email)
        print("Modules switched on:", ", ".join(done) or "all were already on")
    finally:
        db.close()
```

Execution Command:

```
.venv\Scripts\python dev_make_admin.py user@ctdtechs.com
```

5. Environment Verification Checklist

To streamline future environment setups and prevent the recurrence of these configuration hurdles, developers should follow this sequential checklist during initial project provisioning:

- **Step 1: Environment File Formatting:** Ensure .env contains clear newline breaks between all keys, specifically verifying database connection strings and SECRET_BACKEND.
- **Step 2: Database Role Alignment:** Define distinct connection strings for migrations (postgres superuser) and application runtime (hr_app role).
- **Step 3: Schema Migrations & Baseline Seeding:** Execute alembic upgrade head across both primary and control plane databases, followed by tenant initialization and pipeline seeding.
- **Step 4: Execute Role Privileges Grant:** Run grant_app.py to ensure hr_app holds connect, schema usage, table CRUD, and sequence permissions.
- **Step 5: Admin Provisioning & Module Activation:** Execute dev_make_admin.py against target developer emails to grant Super Admin access and activate all system modules.